

DepReduce

Automated Dependency Optimization for Artifact-Based Build Systems

Artifact Evaluation

Artifact for “Automated Dependency Optimization for Artifact-Based Build Systems”. Reviewers should use the prebuilt Docker archives; image builds are author-only.

The main archive, `depreduce-artifact-issta2026.tar.gz`, contains the tool, examples, paper experiment scripts, paper-project data, and precomputed plots.

Getting Started (Estimated 10 minutes)

```
docker load -i depreduce-artifact-issta2026.tar.gz
docker run --rm -it depreduce-artifact:issta2026
```

Inside the container:

```
cd /artifact/depreduce
depreduce --help
depstat --help
bazel --version
buck2 --version
```

Expected output:

```
Usage: depreduce [OPTIONS] --workspace <WORKSPACE>

Options:
  -w, --workspace <WORKSPACE>

  -c, --command <COMMAND>
        [default: ${workspace}/build.sh]
  -t, --target <TARGET>
        Target to query dependencies for [default: //...]
  -o, --output <OUTPUT>
        Output directory for reduction attempts and dep graph [default: logs/]
  -b, --build-system <BUILD_SYSTEM>
```

```

    Build system to use (currently supports: bazel, buck, cargo) [default: bazel]
--config <CONFIG>
    [default: depreduce.toml]
--disable-dependency-flattening
    Disable dependency flattening: prevents the reducer from adding dependencies of the node
being optimized to the dependent node being reduced as dependencies
--enable-dependency-flattening-for-alias-targets
    Enable dependency flattening for alias targets. Disabled by default to avoid flattening
the alias targets because they are usually used to simplify the dependency names or combine
multiple dependencies as a whole.
--disable-dependency-lifting
    Disable dependency lifting: prevents the reducer from adding the node being optimized to
the dependents of the dependent node being reduced as a dependency
--disable-topological-sorting
    Only can be set when disable_dependency_flattening and disable_dependency_lifting are
both set
--enable-optimization-if-transitive-deps-exists
    Also consider to remove a dependency even if it can still be accessed transitively.
Disabled by default to avoid removing direct dependencies.
-h, --help
    Print help
-V, --version
    Print version

Usage: depstat [OPTIONS] [COMMAND]

Commands:
  parse          Parse depreduce logs to collect statistics
  compute-rebuild Compute rebuild set
  help          Print this message or the help of the given subcommand(s)

Options:
  -w, --workspace <WORKSPACE>    [default: .]
  -t, --target <TARGET>          [default: //...]
  -b, --build-system <BUILD_SYSTEM> Build system to use (currently supports: bazel, buck, cargo)
[default: bazel]
  -h, --help                    Print help
  -V, --version                Print version

bazel 8.3.1
buck2 2026-06-11-b940e81e7287c20131248edcc25c93c4e56cc8b9

```

Now, try running DepReduce on the simple C++ Bazel example:

```

cp -a examples/simple-cxx-project /tmp/depreduce-simple-bazel
cat > /tmp/depreduce-simple-bazel/build.sh <<'EOF'
set -e
bazel build //...
EOF
chmod +x /tmp/depreduce-simple-bazel/build.sh

depreduce \
  --build-system bazel \
  -w /tmp/depreduce-simple-bazel \
  -c /tmp/depreduce-simple-bazel/build.sh \
  --config depreduce.toml \
  --output /tmp/depreduce-simple-bazel/logs
echo $?

```

Expected output:

```

Starting reduction test at 2026-07-07T22:31:08.936581221+00:00
Workspace root: "/tmp/depreduce-simple-bazel"
Build script: "/tmp/depreduce-simple-bazel/build.sh"
Args: Args {
  workspace: "/tmp/depreduce-simple-bazel",
  command: "/tmp/depreduce-simple-bazel/build.sh",
  target: "//...",
  output: "/tmp/depreduce-simple-bazel/logs",
  build_system: "bazel",
  config: "depreduce.toml",
  disable_dependency_flattening: false,
  enable_dependency_flattening_for_alias_targets: false,
  disable_dependency_lifting: false,
  disable_topological_sorting: false,
  enable_optimization_if_transitive_deps_exists: false,
}
Starting local Bazel server (8.3.1) and connecting to it...
no actions running
no actions running
no actions running
Loading: 0 packages loaded
Read 0 lines from bazel query output...
Parsed dep graph
Original rebuild cost: 6
Skipping `from` nodes for removal (2): {

```

```

"@rules_cc//:link_extra_libs",
"@rules_cc//:link_extra_lib",
}
Skipping `to` nodes for removal (0): {}
Skipping `from` nodes for addition (0): {}
Skipping `to` nodes for addition (0): {}
Nodes:
0: //main:main (NodeProps { t: Target(TargetType { is_alias: false }) })
1: @rules_cc//:link_extra_lib (NodeProps { t: Target(TargetType { is_alias: true }) })
2: @rules_cc//:link_extra_libs (NodeProps { t: Target(TargetType { is_alias: true }) })
3: //libb:libb (NodeProps { t: Target(TargetType { is_alias: false }) })
4: //liba:liba (NodeProps { t: Target(TargetType { is_alias: false }) })
Unremovable edges:
Running build command: /tmp/depreduce-simple-bazel/build.sh (cwd: /tmp/depreduce-simple-bazel)
Computing main repo mapping:
Loading:
Loading: 0 packages loaded
Analyzing: 3 targets (0 packages loaded, 0 targets configured)
Analyzing: 3 targets (0 packages loaded, 0 targets configured)
INFO: Analyzed 3 targets (64 packages loaded, 489 targets configured).
[11 / 12] [Prepa] Linking main/main
INFO: Found 3 targets...
INFO: Elapsed time: 1.573s, Critical Path: 0.28s
INFO: 5 processes: 8 action cache hit, 2 internal, 3 processwrapper-sandbox.
INFO: Build completed successfully, 5 total actions
Triage build: Build succeeded
Processing node: //main:main (1/5)
Processing node: @rules_cc//:link_extra_lib (2/5)
Trying a new candidate. Remaining candidates: 0
Only consider deps for //main:main -> @rules_cc//:link_extra_lib (because in-degree = 0)
Failed to remove //main:main -> @rules_cc//:link_extra_lib: Dependency Label
'@rules_cc//:link_extra_lib' not found
No changes made, skipping build
Processing node: @rules_cc//:link_extra_libs (3/5)
Trying a new candidate. Remaining candidates: 0
Skipping removing @rules_cc//:link_extra_lib -> @rules_cc//:link_extra_libs (skipped by `from`
rules in config)
Processing node: //libb:libb (4/5)
Trying a new candidate. Remaining candidates: 0
Only consider deps for //main:main -> //libb:libb (because in-degree = 0)
Removed //main:main -> //libb:libb
Running build command: /tmp/depreduce-simple-bazel/build.sh (cwd: /tmp/depreduce-simple-bazel)
Computing main repo mapping:

```

Loading:

Loading: 0 packages loaded

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

INFO: Analyzed 3 targets (1 packages loaded, 2 targets configured).

INFO: Found 3 targets...

INFO: Elapsed time: 0.585s, Critical Path: 0.30s

INFO: 4 processes: 3 action cache hit, 2 internal, 2 processwrapper-sandbox.

INFO: Build completed successfully, 4 total actions

In-degree of //libb:libb is now 0

Committed changes: Build succeeded

Processing node: //liba:liba (5/5)

Trying a new candidate. Remaining candidates: 1

Only consider deps for //libb:libb -> //liba:liba (because in-degree = 0)

Removed //libb:libb -> //liba:liba

Running build command: /tmp/depreduce-simple-bazel/build.sh (cwd: /tmp/depreduce-simple-bazel)

Computing main repo mapping:

Loading:

Loading: 0 packages loaded

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

INFO: Analyzed 3 targets (1 packages loaded, 3 targets configured).

INFO: Found 3 targets...

INFO: Elapsed time: 0.168s, Critical Path: 0.02s

INFO: 2 processes: 2 action cache hit, 1 internal, 1 processwrapper-sandbox.

INFO: Build completed successfully, 2 total actions

In-degree of //liba:liba is now 1

Committed changes: Build succeeded

Trying a new candidate. Remaining candidates: 0

Only consider deps for //main:main -> //liba:liba (because in-degree = 0)

Removed //main:main -> //liba:liba

Running build command: /tmp/depreduce-simple-bazel/build.sh (cwd: /tmp/depreduce-simple-bazel)

Computing main repo mapping:

Loading:

Loading: 0 packages loaded

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

Analyzing: 3 targets (1 packages loaded, 0 targets configured)

INFO: Analyzed 3 targets (1 packages loaded, 2 targets configured).

ERROR: /tmp/depreduce-simple-bazel/main/BUILD:3:10: Compiling main/main.cpp failed: (Exit 1): gcc failed: error executing CppCompile command (from target //main:main) /usr/bin/gcc -U_FORTIFY_SOURCE -fstack-protector -Wall -Wunused-but-set-parameter -Wno-free-nonheap-object -fno-

```

omit-frame-pointer '-std=c++17' -MD -MF ... (remaining 24 arguments skipped)
    Use --sandbox_debug to see verbose messages from the sandbox and retain the sandbox build
    root for debugging
    main/main.cpp:3:10: fatal error: a.h: No such file or directory
      3 | #include "a.h"
        |           ^~~~~
    compilation terminated.
    Use --verbose_failures to see the command lines of failed build steps.
    INFO: Elapsed time: 0.182s, Critical Path: 0.05s
    INFO: 3 processes: 2 action cache hit, 3 internal.
    ERROR: Build did NOT complete successfully
    Build failed with exit code 1

    Trying to lift dependency node //liba:liba to //main:main
    No in-edges for //main:main -> //liba:liba, skipping lift
    Trying to flatten dependencies for node //liba:liba
    No changes made, skipping build
    Restoring backups:
      /tmp/depreduce-simple-bazel/main/BUILD
    End reduction test at 2026-07-07T22:31:17.175148528+00:00
    Loading: 0 packages loaded
    Read 0 lines from bazel query output...
    Rebuild cost: 6 -> 4
    0

```

As long as there is “Rebuild cost: 6 -> 4” in the output, the reduction is successful.

Then, inspect the logs:

```

ls /tmp/depreduce-simple-bazel/logs
head -5 /tmp/depreduce-simple-bazel/logs/01-attempts.jsonl

```

Expected output:

```

00-graph.json 01-attempts.jsonl
{"candidates":{"node_id":3,"dependent":2},"ops":[]}
{"candidates":{"node_id":4,"dependent":3},"ops":[]}
{"candidates":{"node_id":1,"dependent":2},"ops":[{"Backup":{"path":"/tmp/depreduce-simple-bazel/main/BUILD"}}, {"Apply":{"edit":{"path":"/tmp/depreduce-simple-bazel/main/BUILD","desp":"Remove dependency '//libb:libb' from label '//main:main'"}}, {"Build":{"exit_code":0,"stdout":"","stderr":"Computing main repo mapping: \nLoading: \nLoading: 0 packages loaded\nAnalyzing: 3 targets (1 packages loaded, 0 targets configured)\nAnalyzing: 3 targets (1

```

```

packages loaded, 0 targets configured)\n\nINFO: Analyzed 3 targets (1 packages loaded, 2 targets
configured).\nINFO: Found 3 targets...\nINFO: Elapsed time: 0.585s, Critical Path: 0.30s\nINFO: 4
processes: 3 action cache hit, 2 internal, 2 processwrapper-sandbox.\nINFO: Build completed
successfully, 4 total actions\n"}}, {"Remove": {"node_id": 1, "dependent_node_id": 2}}, {"Commit":
{"paths": ["/tmp/depreduce-simple-bazel/main/BUILD"]}}}]
{"candidates": {"node_id": 0, "dependent": 1, "ops": [{"Backup": {"path": "/tmp/depreduce-simple-
bazel/libb/BUILD"}}, {"Apply": {"edit": {"path": "/tmp/depreduce-simple-
bazel/libb/BUILD", "desp": "Remove dependency '//liba:liba' from label '//libb:libb'}}}], {"Build":
{"exit_code": 0, "stdout": "", "stderr": "Computing main repo mapping: \nLoading: \nLoading: 0 packages
loaded\nAnalyzing: 3 targets (1 packages loaded, 0 targets configured)\nAnalyzing: 3 targets (1
packages loaded, 0 targets configured)\n\nINFO: Analyzed 3 targets (1 packages loaded, 3 targets
configured).\nINFO: Found 3 targets...\nINFO: Elapsed time: 0.168s, Critical Path: 0.02s\nINFO: 2
processes: 2 action cache hit, 1 internal, 1 processwrapper-sandbox.\nINFO: Build completed
successfully, 2 total actions\n"}}, {"Remove": {"node_id": 0, "dependent_node_id": 1}}, {"Commit":
{"paths": ["/tmp/depreduce-simple-bazel/libb/BUILD"]}}}]
{"candidates": {"node_id": 0, "dependent": 2, "ops": [{"Backup": {"path": "/tmp/depreduce-simple-
bazel/main/BUILD"}}, {"Apply": {"edit": {"path": "/tmp/depreduce-simple-
bazel/main/BUILD", "desp": "Remove dependency '//liba:liba' from label '//main:main'}}}], {"Build":
{"exit_code": 1, "stdout": "", "stderr": "Computing main repo mapping: \nLoading: \nLoading: 0 packages
loaded\nAnalyzing: 3 targets (1 packages loaded, 0 targets configured)\nAnalyzing: 3 targets (1
packages loaded, 0 targets configured)\n\nINFO: Analyzed 3 targets (1 packages loaded, 2 targets
configured).\nERROR: /tmp/depreduce-simple-bazel/main/BUILD:3:10: Compiling main/main.cpp failed:
(Exit 1): gcc failed: error executing CppCompile command (from target //main:main) /usr/bin/gcc -
U_FORTIFY_SOURCE -fstack-protector -Wall -Wunused-but-set-parameter -Wno-free-nonheap-object -fno-
omit-frame-pointer '-std=c++17' -MD -MF ... (remaining 24 arguments skipped)\n\nUse --
sandbox_debug to see verbose messages from the sandbox and retain the sandbox build root for
debugging\nmain/main.cpp:3:10: fatal error: a.h: No such file or directory\n    3 | #include
\"a.h\"\n        |          ^~~~~\ncompilation terminated.\nUse --verbose_failures to see the
command lines of failed build steps.\nINFO: Elapsed time: 0.182s, Critical Path: 0.05s\nINFO: 3
processes: 2 action cache hit, 3 internal.\nERROR: Build did NOT complete successfully\n"}},
{"Restore": {"paths": ["/tmp/depreduce-simple-bazel/main/BUILD"]}}}]

```

Inspect the build-file edit:

```

git diff --no-index \
  examples/simple-cxx-project/main/BUILD \
  /tmp/depreduce-simple-bazel/main/BUILD || true

```

Expected output:

```

diff --git a/examples/simple-cxx-project/main/BUILD b/tmp/depreduce-simple-bazel/main/BUILD
index df18cce..c6ee986 100644

```

```

--- a/examples/simple-cxx-project/main/BUILD
+++ b/tmp/depreduce-simple-bazel/main/BUILD
@@ -7,6 +7,6 @@ cc_binary(
    ],
    deps = [
        "//liba",
-       "//libb",
+
    ],
)

```

Also inspect another build-file edit:

```

git diff --no-index \
    examples/simple-cxx-project/libb/BUILD \
    /tmp/depreduce-simple-bazel/libb/BUILD || true

```

```

diff --git a/examples/simple-cxx-project/libb/BUILD b/tmp/depreduce-simple-bazel/libb/BUILD
index fab2a75..ac8ab7c 100644
--- a/examples/simple-cxx-project/libb/BUILD
+++ b/tmp/depreduce-simple-bazel/libb/BUILD
@@ -7,6 +7,6 @@ cc_library(
    includes = ["."],
    visibility = ["//visibility:public"],
    deps = [
-       "//liba",
+
    ],
)

```

Explanation: `//libb` is removed from `//main:main`, and `//liba` is removed from `//libb:libb`, while the Bazel build still passes.

Reproduce Paper Analyses from Bundled Data

```

python3 scripts/stats/analyze_builds.py
python3 scripts/stats/analyze_clean_build.py \
    --data-root data/experiment \
    --out-dir /tmp/depreduce-clean-plots

```

The precomputed publication plots and clean-build summary are also included:


```
ls plots
cat plots/clean_build_summary.md
```

Expected output:

```
clean_build_summary.md  scatter_cost_vs_improvement_faceted.pdf
scatter_executed_actions_vs_cost_faceted.pdf

| Project | Before (median  $\pm$ 95%CI ms) | After (median  $\pm$ 95%CI ms) | Improvement (ms) | Improvement (%) | runs (B/A) |
|---|---:|---:|---:|---:|---:|
| hirschgarten | 152839 [146745, 170767] | 149375 [143799, 152319] | 3464 | 2.3% | 5/5 |
| angular | 236887 [230592, 240748] | 232207 [228526, 240788] | 4680 | 2.0% | 5/5 |
| buildfarm | 185347 [182869, 185996] | 182003 [180940, 186500] | 3344 | 1.8% | 5/5 |
| gerrit | 71874 [69235, 80016] | 71582 [69875, 73360] | 292 | 0.4% | 5/5 |
| copybara | 275574 [273535, 276980] | 277116 [276363, 278259] | -1542 | -0.6% | 5/5 |
```

Useful data paths: `data/experiment/*/*-output/01-attempts.jsonl`, `data/experiment/*/*-rebuild*.json`, and `data/experiment/*/stats/{incre_build,clean_build}`.

Optional Zirgen Case Study Image (Estimated 2-3 hours)

The optional archive, `depreduce-artifact-zirgen-issta2026.tar.gz`, adds a real-world project case study with [Zirgen](#) checkout at the paper baseline.

```
docker load -i depreduce-artifact-zirgen-issta2026.tar.gz
docker run --rm -it depreduce-artifact-zirgen:issta2026
```

Inside the container:

```
cd /artifact/zirgen
git reset --hard df6fb9dda1c20209058d6ee90a8912351b741081
git clean -fd
```

Prepare the build script:

```
cat > /tmp/zirgen-full-build.sh <<'EOF'
set -e
cd /artifact/zirgen
bazel build //zirgen/dsl:zirgen
```

```
bazel test --notest_keep_going //...
EOF
```

Run DepReduce:

```
depreduce \
  --build-system bazel \
  -w /artifact/zirgen \
  -c /tmp/zirgen-full-build.sh \
  --config /artifact/depreduce/scripts/experiments/zirgen.toml \
  --output /tmp/zirgen-full-depreduce-output
```

Summarize the DepReduce log:

```
depstat parse -l /tmp/zirgen-full-depreduce-output/
```

Expected Zirgen summary: `n_removals: 24`, `n_lifting: 10`, `n_flattening: 5`.

It is possible that the numbers differ from the paper due to build flakiness.

Full Experiment Reruns (Not Bundled)

Scripts are under [scripts/experiments](#), [scripts/buck](#), [scripts/rust](#), and [scripts/stats](#). Full real-project reruns require external checkouts, network access, project dependencies, and substantial runtime.

Author Packaging Commands (For Developer Reference; Not for Reviewers)

```
git submodule update --init
docker build -t depreduce-artifact:issta2026 .
docker save depreduce-artifact:issta2026 | gzip -1 > depreduce-artifact-issta2026.tar.gz
sha256sum depreduce-artifact-issta2026.tar.gz > depreduce-artifact-issta2026.tar.gz.sha256

docker build -f Dockerfile.zirgen -t depreduce-artifact-zirgen:issta2026 .
docker save depreduce-artifact-zirgen:issta2026 | gzip -1 > depreduce-artifact-zirgen-
issta2026.tar.gz
sha256sum depreduce-artifact-zirgen-issta2026.tar.gz > depreduce-artifact-zirgen-
issta2026.tar.gz.sha256
```

Paper Claims Supported by This Artifact

Paper claim	Artifact support	How to evaluate
DepReduce edits dependency declarations and validates changes by rebuilding.	Supported by code and example run.	Run Getting Started; inspect generated <code>01-attempts.jsonl</code> and diffs.
DepReduce supports Bazel, Buck, and Cargo.	Supported by implementation, examples, and scripts.	Run tool help; inspect <code>depreduce/src/supports/</code> and example scripts.
DepReduce performs direct removal, lifting, and flattening.	Supported by reducer code and bundled logs.	Inspect <code>depreduce/src/reducers/</code> and <code>data/experiment/*/*-output/01-attempts.jsonl</code> .
Paper Bazel evaluation and aggregate results.	Supported by paper-project scripts plus available bundled outputs.	Inspect <code>scripts/experiments</code> , <code>data/experiment</code> , and run analysis scripts.
Build-time/action-count analyses.	Supported by bundled build-event logs and plots.	Run <code>scripts/stats/analyze_builds.py</code> and <code>analyze_clean_build.py</code> ; inspect <code>plots/</code> .
DepReduce results on Zirgen.	Supported by optional image with Zirgen checkout, build script, and bundled log.	Run the optional Zirgen section; inspect the summary and compare to paper.

Paper Claims Not Fully Supported by This Artifact

Claim or result	Why it is not fully supported in the packaged artifact
Full rerun of all real-project experiments during review.	Requires external checkouts, network access, project-specific dependencies, and long runtimes.

Recommended Reviewer Scope

Default 30-minute review: run Getting Started and bundled-data analyses.

Deeper (may take hours): load the Zirgen image, run DepReduce, and summarize the bundled Zirgen log.

Overview

Currently supports the following build systems:

- Bazel
- Buck
- Cargo (for Rust)

See `depreduce` for implementation details.

How to Use `depreduce`

```
depreduce \  
  -w <WORKSPACE_DIR> \  
  -c <BUILD_SCRIPT> \  
  --config <depreduce.toml> \  
  --build-system <bazel|buck|cargo>
```

See scripts in [scripts/experiments](#) or [scripts/buck](#) for more examples.

Example Build Script

```
set -e  
bazel build //...  
bazel test --notest_keep_going -- //...
```

Example `depreduce.toml`

See [depreduce.toml](#)

Extensions to Related Tools

This part is out of the scope of the artifact evaluation or our paper, but we include it here for completeness and to share our efforts in applying related tools to Bazel.

We also extend two related tools to support Bazel build system and investigate their effectiveness on redundant dependency detection. It turns out that neither of them can do it well.

Extending `mkcheck` to `buildfuzz`

`buildfuzz` is the reproduction of the build fuzzing algorithm proposed by `mkcheck` (<https://github.com/nandor/mkcheck>) with new features:

1. Use **custom touchers** instead of the `touch` file operation, which will **CHANGE** the file content but not affect the original functionality.

2. Use **SHA256** instead of timestamp to detect file changes.
3. **Restore** touched file content after every round.
4. **Rebuild** the project before every round.

How to Run `buildfuzz`

```
buildfuzz --input examples/simple-cxx-project \  
  --artifact examples/simple-cxx-project/bazel-bin \  
  --command buildfuzz/src/test_data/build.sh \  
  --output result_deps.log
```

The result is a JSONL file like below.

```
["a.o",["a.h","a.c"]]  
["b.o",["b.h","b.c"]]  
["main.o",["main.c","a.h","b.h"]]  
["main",["main.o","a.o","b.o"]]
```

Extending `buildfs` to `strace_parser`

`strace_parser` is the reproduction of `buildfs` (<https://github.com/theosotr/buildfs>) with some improvements:

1. **stat / lstat / statfs syscalls were ignored** because we don't know if the accessed file truly exists, and they are mostly used to detect file changes, i.e., usually not a real sign of file consumption.
2. **Syscalls returning -1** will be ignored, because they failed mostly for inexistent files.
3. **clone3 syscall was added** for tracing. Otherwise there will be many missing `Newproc` operations.
4. **--decode-pids=pids,comm was added** as the arguments of `strace`, to resolve the pid within a separate namespace, which is the case of Bazel sandboxing. Otherwise, the pids returned by `clone` or `fork` cannot match the pids traced by `strace` in its own namespace, which prevents us from tracing the process relationship correctly.
5. A **virtual filesystem** was implemented to track symlinks. Bazel creates lots of symlinks because of sandboxing.
6. A **to_link operation was added** to DSL (IR) to support tracking symlinks.

Though `buildfs` claims their approach is applicable to other build systems including Bazel. The fact is, without our efforts, it is really hard to apply it to Bazel. See [Sandboxing - Bazel](#) for details about the sandboxing mechanism in Bazel.

How to Run `strace`

```
bazel clean --expunge
bazel shutdown
strace -s 300 \
    -f \
    -e
access,chdir,chmod,chown,clone,clone3,close,dup,dup2,dup3,execve,fchdir,fchmodat,fchownat,fcntl,fo
rk,getxattr,getcwd,lchown,lgetxattr,lremovexattr,lsetxattr,link,linkat,mkdir,mkdirat,mknod,open,op
enat,readlink,readlinkat,removexattr,rename,renameat,rmdir,symlink,symlinkat,unlink,unlinkat,utime
,utimensat,utimes,vfork,write,writew \
    --decode-pids=pids,comm \
    -o strace.log \
    bash ../build_for_strace.sh
```

How to use `strace_parser`

```
strace_parser -i examples/simple-java-project/strace.log -c examples/simple-java-project -o
result_deps.log
```

The result is a JSONL file like below.

```
["a.o",["a.h","a.c"]]
["b.o",["b.h","b.c"]]
["main.o",["main.c","a.h","b.h"]]
["main",["main.o","a.o","b.o"]]
```